P1604R1
The inline keyword is not in line with the design of modules.

Published Proposal, 2019-06-16

Author:
Corentin Jabot

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The `inline` keywords make little sense in modules units. We, therefore, propose to change its meaning in module unit contexts.

Table of Contents

§ 1. Introduction

`inline` was intended as a hint for the compiler that performing inlining optimizations of marked functions was desirable. However, for inlining to be possible, the definition of functions must be

visible to the compiler - aka a prerequisite of inlinable functions is to have their definition visible in the TUs in which they are called.

To make function definitions visible in a non-modular world, it is customary to define them in headers. However, headers are often included (stitched) to multiple sources files thereby forming multiple TU containing duplicated definitions of the same functions.

This would, of course, cause linkage error, and so inline gained a new semantic:

> An inline function or variable shall be defined in every translation unit in which it is odr-used and shall have exactly the same definition in every case.

The inlining semantics of `inline` is described in this delightful normatively non-normative clause:

> The inline specifier indicates to the implementation that inline substitution of the function body at the point of call is to be preferred to the usual function call mechanism. An implementation is not required to perform this inline substitution at the point of call;

And so `inline` means both "This function is a candidate for inlining" and "This function might be defined multiple times". This duality has been a constant source of confusion as developers struggle to understand what `inline` does and why both semantics are orthogonal yet intertwined. A quick "C++ inline" search on the internet reveals how poorly `inline` is understood.

Or is it?

Practices such has header-only libraries make the link behavior of `inline` much more prevalent than the inlining hint semantic. In such scenarios, functions are marked `inline` that are poor candidates for inlining. It is also not easy to determine how much compilers rely on `inline` as an inlining hint [inline-hint].

§ 2. Inline and Modules

The module proposal [P1103r3] states:

> A module unit is a translation unit that forms part of a module.

Specifically, this implies that module interface units are translation units, rather than text files stitched together to form translation units.

This is a fundamental property of modules and one that makes them a superior alternative to non-modularized code and legacy header units.

And so, non-inline symbols defined in module interface units have a guaranteed unique definition. This solves a number of ODR violation issues and as such make code more reliable, which again is a major selling point of modules - Especially given that ODR violations are very hard or impossible to properly diagnostic.

By materializing interfaces as an actual entity, modules further make it easier to reason about code ownership, API and ABI, as each exported symbol is tied to an interface, which has a name.

So, what is the purpose of `inline` in module-unit context?

Since modules units form translation units, the compiler has a unique place to put and find the compiled symbols of all symbols declared in a module unit and do not need to duplicate any symbols declared in modules units. But symbols marked `inline` need to be defined in all TUs in which they are used, even in module units.

This has undersirable consequences:

§ 2.1. Compile time pessimizations

Machine code needs to be generated for `inline` functions in every translation unit in which they are used. While arguably generating code for functions is fast, duplicating that work over a large number of TUs adds up. Given that compilation, speed is an ever increasing issue and that modules have been branded as a mean to reduce compilation time significantly, it would prove beneficial to reduce the amount of duplicated work a compiler has to do to compile a program.

Compiling 1000 TUs calling the same simple function `f` revealed a 10-25% difference (varying across compilers and optimizations levels) depending on whether `f` was marked `inline` or defined in a separate TU. While gains in real code or in more thorough tests would be less pronounced, they would still be noticeable.

Of course, when `inline` functions are actually inlined, compilation times are not impacted by whether or not a method is redefined - inlining is desirable and does not constitute work duplication.

§ 2.2. ODR violations

`inline` allows a symbol to be redefined multiple time but mandates that each definition must be identical. However, we don’t have the tools to properly diagnostic or prevent violations of this rule,

and as such `inline` in module interface units might be the source of ODR violations, despite modules being branded as a tool to limit or better diagnostic such issues.

§ 2.3. Inlining of non `inline` functions

Because the way modules interface units are imported is implementation defined, whether the definitions defined in module interface units are visible (such that inlining can be performed) is equally implementation-defined. To be more precise

- `inline` and implicitly inline definitions are always visible.
- Whether non-inline definitions (exported or not) can be inlined in importing TUs is implementation-defined.

In practice, we observe that different compilers have different policies as to whether non-inline definitions are included in binary modules interfaces.

This is a confusing departure from the header-modules in which the definitions of all symbols defined in headers were visible and therefore candidates for inlining.

- Exporting a symbol is not sufficient to make its definition portably visible across compilers
- Not marking a symbol inline is not sufficient to hide its definition portably visible across compilers

Implementers have suggested relying on Link Time Optimization to perform inlining of non-inlined symbols, however we believe this to be an unsatisfactory workaround as LTO is usually time and resource consuming, not universally used and not as efficient as compile-time inlining.

§ 3. Proposed solutions

§ 3.1. Separate the "inlinable" and "can have multiple definitions" definitions of `inline`

We suggest that, in the purview of a module unit, `inline` should only mean inlinable.

This would allow a wider range of implementation strategies, including not re-defining a definition in every Translation Units.

With this change, it would be implementation-defined whether an inline entity is redefined in multiple TU.

At the same time, the standard would allow (but not force) currently "implicitly inline" entities to be defined in more that one translation units

- template instantiations
- default and deleted class members
- class members defined in the class declaration
- implicitly declared class members
- constexpr functions
- lambda call operators
- `constexpr` functions

§ 3.2. No implicit `inline`

In the purview of a module unit, the standard would not implicitly declare any entity as inlinable, including any currently "implicitly `inline`" entity (list above).

§ 3.3. Discourage vendors to `inline` exported functions non explicitly inline.

While inlining is a QoI issue, inlining has ABI implications and it is important that it remains possible to maintain a stable ABI exposed through a module interface.

As such we wish to discourage implementation to inline exported symbols. the C++ ecosystem TR seems the right place to word such discouragement.

§ 4. Alternatives

§ 4.1. Introduce an `[[inline]]` attribute

Because `inline` currently does two things that are intertwined in a too often poorly understood way, and because the proposed changes give `inline` a different meaning in non-modularized code and module unit purviews, it might be interesting to make the `inline` keyword ill-formed entierly while replacing it by an `[[inline]]` attribute. As inlining is purely a QoI concern, inlining is a prime candidate for the use of an attribute. Doing so would make it clearer what `[[inline]]` does and would be easier to teach.

§ 4.2. Should some entities be inlinable by default?

Again, this is a QoI concern, and implementers are always free to inline any definition they desire, but it might be useful to encourage that some entities, notably lambda closures remain implicitly inlinable. Along with that, it might be interesting to offer a standardized mechanism to opt-out of inlining optimization with a `[[noinline]]` attribute - which would standardize an existing practice in MSVC (`__declspec(noinline)`) and GCC (`__attribute__((noinline))`)

§ 5. FAQ

§ 5.1. What about module implementations units and modules partitions?

All modules units should have the same specification when it comes to inline (or lack thereof), and definition visibility.

§ 5.2. What about Preamble, Legacy Header units and Non-Modular code?

For backward compatibility reasons, we do not propose to change the semantics of inline, implicitly inline or non-inline symbols in these contexts.

§ 5.3. How does this compare with [P1498]?

[P1498] propose to deprecate `inline` in implementation units as well as for non-exported symbols.

Furthermore, [P1498] proposes changes as to what methods can be declared `inline` and be inlined. With this proposal, the compiler can decide automatically when a method cannot be inlined (ie when it calls a symbol with module-local linkage), rather than putting that responsibility on users.

§ 5.4. Wouldn't this create a dialect?

No more than `export` is only valid in module units.

§ 5.5. Why this change and why now?

Modules are a profound change to the way people will compile C++ code and it will transform the ecosystem. It is important to make sure Modules offer the right semantics now and for the next few

decades as it will certainly prove impossible to reasonably do these sort of changes post of C++20.

And while modules offer a better compilation model, they do not get rid of artifacts like `inline`, which were the product of the textual inclusion model of `#include` directives. The multiplication of closely-related but not identical concepts, a wide range of implementation-specific behavior and the loaded history of `inline` makes offering a stable API with clear boundary more challenging than it needs to be.

§ 5.6. If C++ did have modules from the start, would the inline keyword exist at all and would he has its current semantic?

;)

§ References

§ Informative References

[INLINE-HINT]
Simon Brand. Do compilers take inline as a hint?. URL: https://blog.tartanllama.xyz/inline-hints/

[P1103r3]
P1103R3 Merging modules. URL: https://wg21.link/p1103r3

[P1498]
Chandler Carruth; Nathan Sidwell; Richard Smith. Constrained Internal Linkage for Modules. URL: http://wg21.link/p1498